Requirements gathering

What problem are you solving?

Some people spend more than they make in a month. This product will hopefully allow people to track which expenses drain their income and visualise when their income for the month will run out.

Who are your users?

- Money conscious people who want to track whether their income is being spent appropriately. Are they overextending themselves? Can they support their current lifestyle?
- Product should be straightforward and easy to use.

What must it do vs. what's nice to have?

Must Do:

- Log income and expenses correctly.
- Track spendings
- Categorise each transaction as either a necessity or liability (user selects).
- Visualise accurate insight to user's financial situation
- Expenses that apply timely should be applied and presented properly (overlapping cycles)
- Forecast updates in real time when new income or expenses are added.
- Correctly simulate overlapping expense cycles day by day and surface the exact date the user's balance is lowest.
- Forecast view – graph that shows the flow of which the user's income is drained.

Nice to have:

- Mobile Version
- Currency Exchange Rate
- Multiple Languages
- Dark and Light mode
- Consider tax

Pages:

1. Forecast View — the main view, the chart, recent transactions

2. Income — where user logs income sources and their schedule

3. Expenses — where user logs expenses, categories, cycle frequency

System design

Tech Stack:

- Frontend: React + TypeScript
- Backend: FastAPI
- Database: PostgreSQL

Database Schema (Initial)

User

- user_id (PK) INT
- firstname VARCHAR(50)
- lastname VARCHAR(50)

Category

- category_id (PK)
- category_name VARCHAR(50)
- is_necessity BOOLEAN

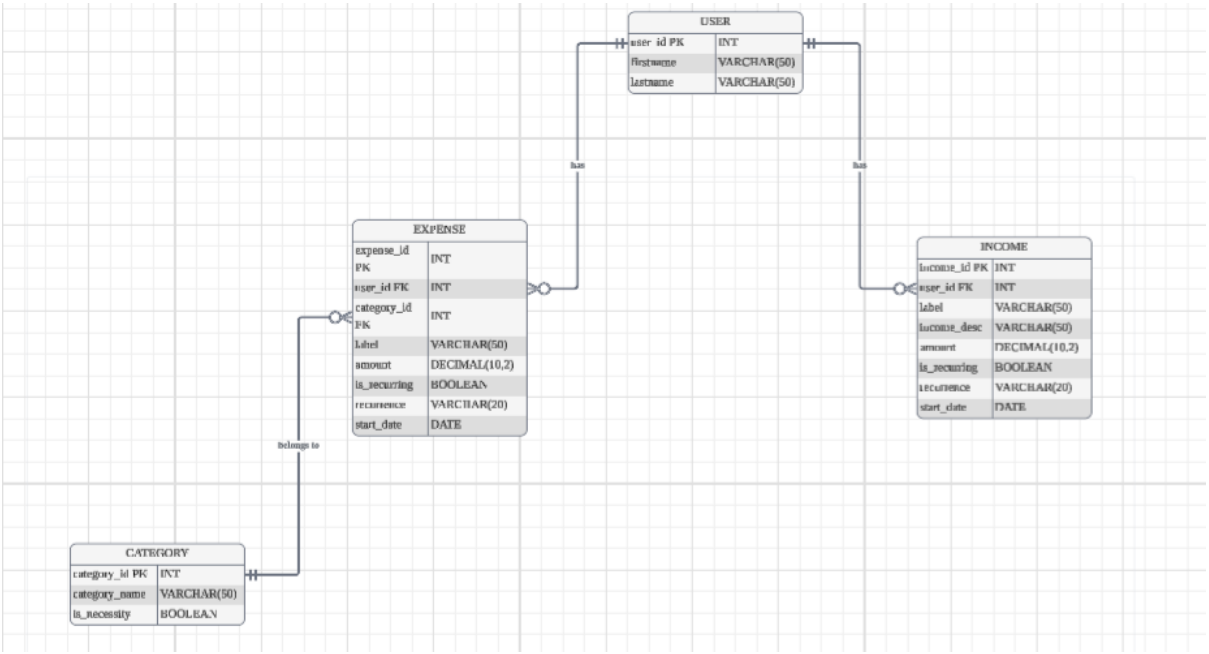
Income

- income_id (PK) INT
- user_id (FK) INT
- income_desc VARCHAR(50)
- amount DECIMAL(10,2)
- is_recurring BOOLEAN
- recurrence VARCHAR(20)
- start_date DATE

Expense

- expense_id (PK) INT
- user_id (FK) INT
- category_id (FK) INT
- label VARCHAR(50)
- amount DECIMAL(10,2)
- is_recurring BOOLEAN
- recurrence VARCHAR(20)
- start_date DATE

ERD Diagram



Architecture Design

1) User enters the data in the frontend. Income: salary RM2000 recurring monthly starting on the 1st of August. Expenses: Wifi bill: RM97 monthly, phone data bill: RM25 renews monthly, Netflix RM30 monthly. React validates the inputs and their data type and send to FastAPI via HTTP POST request.

2) FastAPI receives and validates the incoming data again and stores in PostgresSQL via SQLAlchemy returning a response to frontend that data is saved successfully.

3) When user opens the forecast view, React sends a GET request to FastAPI for the forecast for the month which triggers what I'd like to call the flux engine which does the calculation part of retrieving all the income and expenses of the user from PostgresSQL and simulates every day of that month in each day, any income and expenses will be added or deducted. The engine will create a JSON list of the money left for each day as well as the day when users balance is lowest.

4) FastAPI will send the JSON list to React and feed into Recharts to draw a line graph where x axis is days of the month and y axis is user's balance with the lowest point being marked and labelled.

5) The graph will always be recalculated based on the data stored in PostgresSQL.

Assumptions

- Income entered is post-tax.
- The user is using Malaysia Ringgit only.
- The forecast data is not saved and recalculated fresh from what is stored.

Design Decisions

- There is understandably redundancy between Income and Expense table but kept separate form simplicity when there could have been just one Transaction table.
- When designing and thinking initially, the idea in which users remove their expense or income for whatever reason would affect previous months. A possible solution is another table that stores that month's forecast data that freezes the data when the month ends. I'll come back to this if time permits.
- No login screen and authentication for single user.